

Project Management - Git

Casey Stijlaart

Table of Contents

1. Introduction.....	1
2. Milestones	1
3. Labels	1
4. Issues	1
5. Branches	1
6. Merge Requests (MR).....	1
7. Testing.....	2
8. Git Pipeline.....	2

1. Introduction

To ensure a smooth and efficient workflow, the project employs specific methodologies and tools. Overall the group project and the personal project uses the same approach of project management. However, it's worth mentioning that in the case of personal project, such as those managed by a single individual, the use of Merge Requests approvals and reviews are not implemented due to the nature of the project.

2. Milestones

Milestones are used to mark different phases within the project. Tasks are linked to specific milestones, providing a clear overview of the current status of sub-parts of the project.

3. Labels

Labels are used to track the status of created issues. Different labels are assigned to user stories and tasks to distinguish between different types of work. These labels help maintain a clear overview of items in the backlog, those in progress, under review, and closed.

4. Issues

Issues are categorized into user stories and corresponding tasks. User story issues are linked to their associated tasks to establish clear connections between user stories and their sub-tasks. When creating an issue, it is essential to provide a clear description. Templates are available for different types of issues, including user stories, documents, and bug reports. Furthermore, it's crucial to add the appropriate labels and milestones right away.

Naming: Issues should have short and descriptive names. When an issue is a user story, prefix it with "User Story:."

5. Branches

Branches are used for maintaining version control and enabling collaboration. To ensure good control, there is one main branch called "Development". For each relevant issue, a branch is created and actively worked on. This created branch is derived from the "Development" branch, this to ensure the branch is always up to date with the latest version. Progress is continuously updated, on the working branch, by pushing and pulling to prevent local progress isolation. Meaning that anything that is done is up-to-date. Using this approach makes sure that, if unexpected events occur, no data is lost. Besides this, it allows for the team members to get insight on its progress. For 'committing' the working branch into the "Development" branch, see [Section 6](#)

6. Merge Requests (MR)

As previously mentioned, in group projects, merge request approvals and reviews are integral to

our project management process. However, in personal projects managed by a single individual, such as a solo endeavor, these aspects are not implemented due to the self-driven nature of the work.

Before closing an issue, a merge request must be initiated. A merge request template is utilized to provide clarity to reviewers about what they are reviewing and, if necessary, validate the changes. Merge requests serve as opportunities for feedback, error detection, and overall security of only validated, and if relevant working implementation, changes. They also ensure that no flawed source code or documents are merged into the development branch, providing a level of protection.

Naming: Merge request names should follow the format "Resolve: [the branch name]."

Approval & Reviews: Two approvals are required for a merge request to proceed. These approvals are obtained through the review process. When creating a merge request, two team members must be designated as reviewers. They will provide feedback and grant approval when appropriate.

7. Testing

When software is written, testing must be done to validate the software. There are various manners of testing, each more suitable for one's purpose. Regarding Git, there is focused on unit testing, to be specific GoogleTests. While the created google test cases can give a great insight, it is of importance that the coverage is created as well. Using coverage ensures that all the lines and function are tested properly, and not missed. Ensuring no bugs arise due to poor code coverage.

8. Git Pipeline

Lastly there is made use of the Git Pipeline. The Git pipeline will run the created script at every commit to any of the branches. Within the various jobs are set up, such as the creation of documentation, googletests and cpp check. While there are various jobs, not all jobs are ran during each pipeline. Documentation has no importance to be created on every run, therefore it is only created when there has been a change or when it is manually triggered. The rest of the jobs are always ran. As it is of great importance the written software and test are up to standard, the cpp check and google tests are run. The google test has an extra rule to ensure the coverage is high enough. In our case we set the rule the coverage must at least be 85% for the pipeline to succeed. For more information, see the file 'PP+GP-git-pipeline'.